

P3425R0: Reducing operation-state sizes for subobject child operations

Table of Contents

- [1. Abstract](#)
- [2. Motivation](#)
 - [2.1. Example](#)
 - [2.2. Example - Revisited](#)
- [3. Proposal](#)
 - [3.1. The core protocol](#)
 - [3.2. Adding a helper for child operation-states \(optional\)](#)
 - [3.3. Implementing make_receiver_for\(\)](#)
 - [3.4. Adding a helper for parent operation-states \(optional/future\)](#)
 - [3.5. Applying this optimisation to standard-library sender algorithms](#)
- [4. Design Discussion](#)
 - [4.1. Naming of inlinable receiver concept and inlinable operation state](#)
- [5. Proposed Wording](#)
- [6. References](#)

Document **P3410R0**

Date **2024-10-16**

Reply To **Lewis Baker <lewissbaker@gmail.com>**

Audience **LEWG**

1. Abstract

This paper proposes defining a standard protocol between parent operation-state and child-operation state that permits the child-operation state to avoid needing to store the receiver in cases where the receiver can be computed on-demand from the address of the child operation-state object.

Enabling this optimisation can save a pointer of storage for each nested operation-state, e.g. saving 40 bytes for a sender-expression 5 levels deep. Further, it can improve the performance of queries to retrieve properties from the receiver's environment, by translating a series of pointer-dereferences through successive parent-operation-state pointers into a series of constant-offsets from the child operation-state address, allowing compilers to constant-fold such queries into a single pointer-dereference, reducing code size and improving runtime performance.

Which such protocol is purely additive and could theoretically be added later, there are advantages to adding it now as it permits us to specify that default implementations of standard-library algorithms added by P2300 must incorporate this protocol in their implementation - something that may be difficult to retrofit later, due to the potential for this causing breaking changes in type-layout.

2. Motivation

When a sender-algorithm composes one or more child operations, it typically connects the child senders with a receiver that contains a pointer to the parent operation-state as its only data-member. The operation-state returned from the call to `connect()` will need to store this receiver as a data-member so that it can later call the completion-functions on the receiver to signal completion of the operation.

This means that for every sender in a sender expression-tree we generally have a linked list of operation-states, with pointers wrapped up inside receivers, going from leaf operations up to the top-most operation. This bears similarities to a traditional call-stack, although in the case of senders the links can form a tree rather than being limited to a linear stack.

For algorithms that have a statically-known number of child operations, it is common to have the child operation-states stored as data-members of the parent operation-state as this allows them to avoid dynamic memory allocations.

However, if we think about it, for cases where the child operation-state is a direct member of the parent operation-state, the pointer to the parent operation-state stored in the receiver held by the child is always going to be pointing to some constant offset relative to the child operation-state's address.

In cases where we can compute the address of the parent operation-state from the address of the child operation-state, we can potentially reduce the size of child operation-states by not storing the receiver and instead just constructing the receiver on-demand from the address of the child operation-state whenever the child operation-state needs it.

Doing this would have two main of benefits:

1. It reduces the size of an overall operation-state tree by at least one pointer for each operation in the tree (it might save more due to padding for some operation-states).
2. When querying the environment of the receiver it can turn a walk of a linked-list of operation-states to get the address of the operation-state providing some query result into applying a series of constant offsets from the current operation-state's address.

There are also some secondary benefits for composition of algorithms that adopt these optimisations:

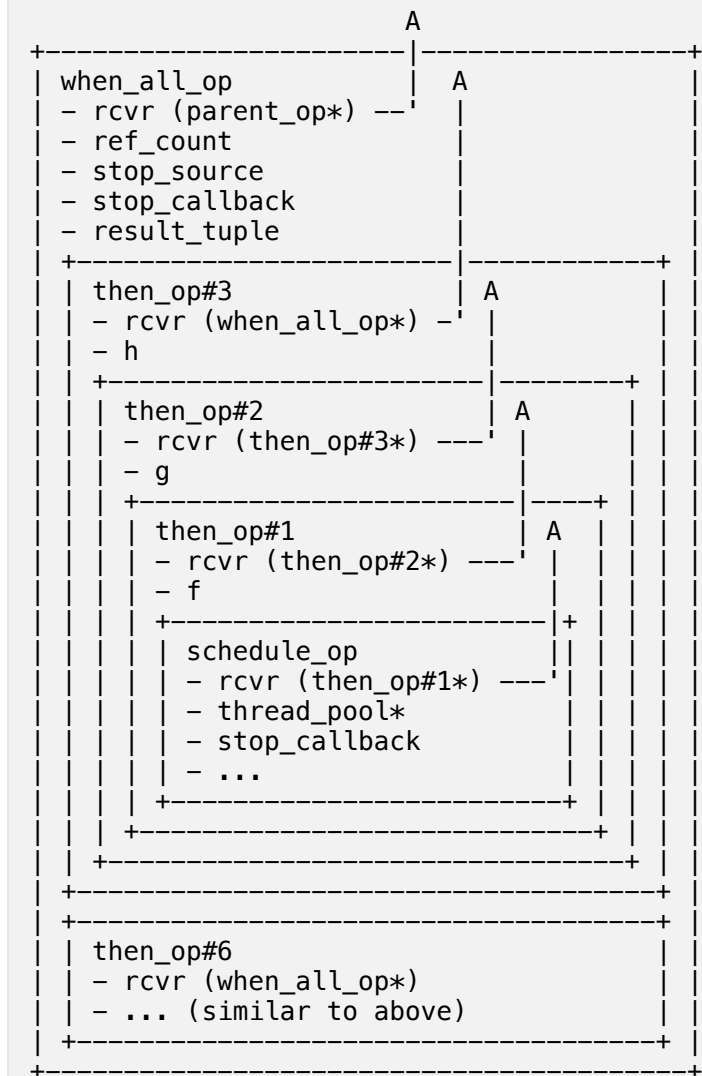
1. It reduces the run-time overhead of composing existing algorithms to build new algorithms.
2. This in turn reduces the need to write your own sender algorithms from scratch as often in order to get optimal performance.

2.1. Example

For example, consider the following sender-expression:

```
when_all(  
  then(  
    then(  
      then(  
        schedule(thread_pool),  
        f),  
      g),  
    h),  
  then(  
    then(  
      then(  
        schedule(thread_pool),  
        a),  
      b),  
    c))  
    // then_op#3  
    // then_op#2  
    // then_op#1  
    // then_op#6  
    // then_op#5  
    // then_op#4
```

With the status-quo, connecting the resulting sender to a receiver, `rcvr`, would result in the following operation-state layout:



There are a few things worth noting here.

Operation State Size

The child operation states all hold a receiver that contains a pointer to the parent operation-state. In total, this consists of 8x pointers to parent operation-states (9x pointers if you include the one likely to be stored in the receiver held by `when_all_op`).

Together, these contribute at least 64-72-bytes in total across the whole operation-state hierarchy - possibly more depending on the size/alignment of the function-objects passed to `then()` if padding is required.

Cost of Environment Queries

The leaf `schedule_op` operations need to subscribe to a stop-callback on the environment's current stop-token in order to support cancellation of the operation - the `when_all()` algorithm can send a stop-request to children if any of them fail.

However, in order to obtain the stop-token needed to register the stop-callback, the `schedule_op` implementation needs to ask its receiver by calling `std::get_stop_token(std::execution::get_env(rcvr))`.

The `get_stop_token` query on the receiver stored in the `schedule_op` forwards the query to the receiver stored in the `then_op#1` object, which then forwards the query to the receiver stored in the `then_op#2` object, which then forwards the query to the receiver stored in the `then_op#3` object, which then satisfies the query by calling `stop_source.get_token()` on the stop-source stored in the `when_all_op` object.

This is 4x pointer dereferences needed to obtain the address of the stop-source object required to evaluate obtain the stop-token. Queries which propagate further up the stack of sender-operations might have to do even more pointer dereferencing to get the query result.

Further, each query that is satisfied by a parent environment up the stack will require its own walk through these pointers to the operation-state that fulfils that particular query.

This stack-walking has a run-time cost - the successive loads from memory of the pointer data cost possibly a few cycles if the data is cache, but could be 10s or 100s of cycles if several of the loads need to go to main memory. The successive loads are all dependent on the prior loads and so the CPU cannot generally pipeline the loads.

The stack-walking logic also has a code-size cost. The compiler needs to generate a sequence of `N` `mov/load` instructions for evaluating each query, where `N` is the number of levels up the stack the query needs to traverse in order to get to the operation-state that satisfies the query.

Cost of Completion

Similar to the cost of pointer-walking for performing queries, calling the completion-function on a receiver also often requires dereferencing the same set of pointers.

In the example above, when the `schedule-operation` completes it needs to load the pointer to the `then_op#3` operation state from the `schedule_op` state in order to compute the address of the function-object, `f`, to invoke. Then, when `f()` returns, it needs to load the pointer to the `then_op#2` operation state from the `then_op#3` state in order to compute the address of the function-object, `g`, to invoke, and so forth.

In a more extreme example, consider the case where a composition of nested operations all just forward through the result to the parent receiver up `N` levels until the eventual ancestor operation that handles the result. Even in this case, where there is no processing of the result datums being done, we still need to follow the linked-list of operation-states in order to compute the address of the final handler of the result.

Cost of Composition

The net result of all of the above costs is that there is a cost to composing these operations.

If the code had, instead, been written with a single `then()` which took a function object that composed `f`, `g` and `h` then the result would have less overhead than the expression where each of these transformations is applied in a separate `then()` invocations.

For example, we could have written:

```
when_all(  
  then(schedule(thread_pool), [f, g, h] { return h(g(f())); }),  
  then(schedule(thread_pool), [a, b, c] { return c(b(a())); }))
```

and this would have fewer operation-state pointers and fewer pointer-indirections than the original code above.

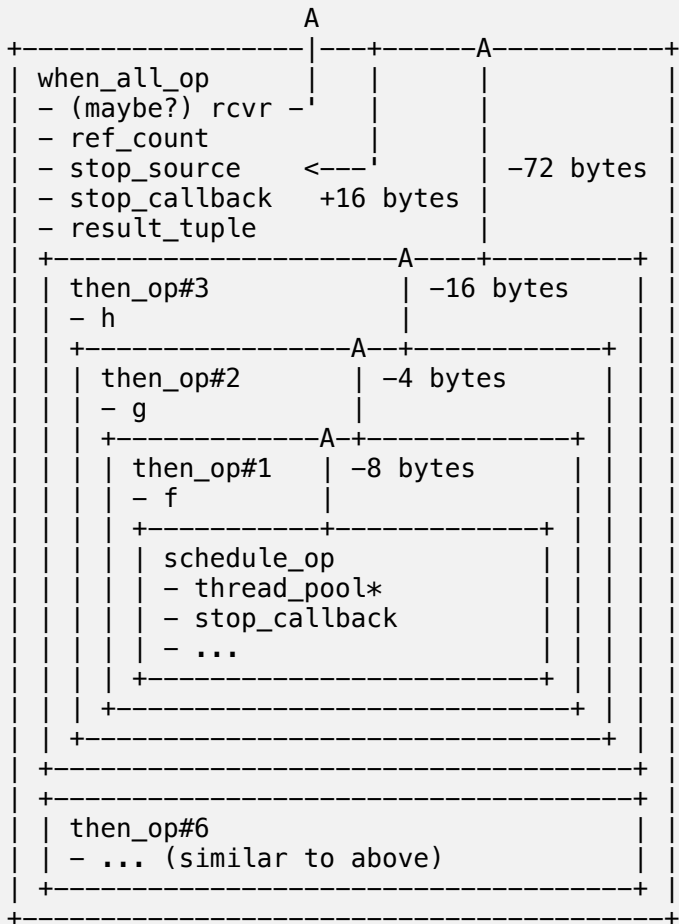
While, in some cases, this kind of manual flattening of composition is possible - it is not always possible.

This makes the cost of composition have non-zero runtime overhead.

This is likely to have the unfortunate side-effect of encouraging users to try to write their code using as few layers of composition as possible - potentially making their code more complex, or even having to write new sender algorithms that implement certain compositions more efficiently.

2.2. Example - Revisited

If we look at the above example, but this time with the optimisations proposed in this paper being applied, then the resulting operation-state will, instead, look something like this:



In this case, each of the child operations knows how to compute the address of the parent-operation state from the address of the child operation state - because the parent operation-state injects this information in along with the receiver in the form of a static function on the receiver type.

So, when the `schedule_op` object goes to construct the `stop_callback` member and needs to get the stop-token from the environment, the compiler sees a series of inlinable calls to compute the parent receiver, each of which just subtracts an offset from the child operation-state.

The net result is that, in optimised compilation modes, the compiler can constant-fold all of these offsets into a single offset from the `schedule_op` address and thus does not need to perform any memory loads in order to obtain the stop-token (which is just initialized with the address of the stop-source object).

For example, in the above operation-state layout diagram, the compiler would effectively lower this code to the equivalent of the following (after inlining):

```

void schedule_op::start() noexcept {
    // Evaluate:
    // auto st = std::get_stop_token(std::get_env(this->get_receiver()));
    //
    // Lowers to equivalent to:
    auto* _op1 = reinterpret_cast<then_op_1*>(reinterpret_cast<unsigned char*>(this) - 8)
    auto* _op2 = reinterpret_cast<then_op_2*>(reinterpret_cast<unsigned char*>(_op1) - 4)
    auto* _op3 = reinterpret_cast<then_op_3*>(reinterpret_cast<unsigned char*>(_op2) - 16)
    auto* _when_all_op = reinterpret_cast<when_all_op*>(reinterpret_cast<unsigned char*>(_op3) - 16)
    auto st = _when_all_op.stop_source.get_token();
    // ...
}

```

Which, after constant-folding would result in a constant offset from this:

```

void schedule_op::start() noexcept {
    // Evaluate:
    // auto st = std::get_stop_token(std::get_env(this->get_receiver()));
    //
    // Lowers to equivalent to:
    auto& _ss = *reinterpret_cast<std::inplace_stop_source*>(
        reinterpret_cast<unsigned char*>(this) - 84);
    auto st = _ss.get_token();
    // ...
}

```

In addition to this being more optimisable by the compiler, the overall operation-state size has now shrunk by at least 64-bytes due to not having to store the pointers to parent-operation states.

There is also now a reduction in code-size in the resulting binary. There are no longer instructions needed to initialize the the pointers to parent-operation-states. There is no longer instructions needed to dereference the chain of pointers during query evaluation or on completion.

The overall net result is that this optimisation permits a reduction in memory usage, an increase in run-time performance and a reduction in code-size proportional to the depth of the sender expression tree that can be inlined.

Further, this code is now more efficient than the hand-flattened version above that combined the three nested invocations of `then()` into a single invocation of `then()`, reducing the motivation for programmers to perform this sort of manual optimisation.

3. Proposal

The proposal includes two key parts which enable the optimisations mentioned above:

1. Defining the key protocol that allows a parent and child operation to negotiate to apply the optimisation when both support it.
2. Applying this protocol to the sender-algorithms proposed by P2300R10.

This proposal also includes some utilities which can be used to make it easier for authors of sender types to implement the above optimisation protocol correctly. These facilities could be optionally included either now or later. If not included now the sender authors can still implement the protocol, but will need to implement their own versions of these helpers in the meantime.

3.1. The core protocol

The key, enabling part of this optimisation is providing a child operation with a way to construct a receiver on-demand given the address of the child operation.

The mechanism proposed here for this is to allow receiver types to define a static factory function that accepts a pointer to the child operation-state and that returns an instance of that receiver type.

For example:

```
struct some_receiver {
    // Factory-construct a receiver on-demand from the child operation-state address.
    static some_receiver make_receiver_for(child_op_state* op) noexcept;

    // Other receiver methods.
    void set_value(auto&&... vs) noexcept;
    void set_error(auto&& e) noexcept;
    void set_stopped() noexcept;
    some_env get_env() const noexcept;
};
```

If the receiver has such a factory function then the child operation is free to not store the receiver passed to `connect()` and to, instead, just call this factory function to obtain a new receiver object whenever the receiver is needed.

This requirement basically defines a new concept that subsumes the receiver concept which can be written as follows:

```
namespace std::execution
{
    template<typename T, typename ChildOp>
    concept inlinable_receiver =
        receiver<T> &&
        requires(ChildOp* op) {
            { T::make_receiver_for(op) } noexcept -> std::same_as<T>;
        };
}
```

Note that the concept does not check that `ChildOp` satisfies `operation_state` as the concept needs to be usable at a point where the `ChildOp` type is still an incomplete type.

With this concept, a child operation-state type, `ChildOp`, can then specialise itself to either hold the receiver as a data-member or not depending on whether the receiver type satisfies the `inlinable_receiver<ChildOp>` concept.

For example:

```
template<typename Receiver>
class my_op_state {
public:
    my_op_state(Receiver r) noexcept : rcvr_(std::move(r)) {}
    void start() noexcept;
private:
    Receiver& get_receiver() noexcept { return rcvr_; }
    Receiver rcvr_;
};

template<typename Receiver>
requires inlinable_receiver<Receiver, my_op_state<Receiver>>
```

```

class my_op_state<Receiver> {
    my_op_state([[maybe_unused]] Receiver r) noexcept {}
    void start() noexcept;
private:
    Receiver get_receiver() noexcept { return Receiver::make_receiver_for(this); }
    // NOTE: No 'Receiver' data-member.
};

```

It is worth noting that the optimisation proposed here requires both the parent operation and child operation to opt-in to the protocol for the optimisation to be applied. If either the parent or child do not opt-in to the protocol then we need to still gracefully revert back to the default behaviour of storing the receiver.

We can see how this would work by examining the code above:

- If the specialisation for an `inlinable_reciever` was not present, as would be the case if the child operation did not opt-in to the optimisation, then the child operation would just store the receiver as normal.
- If the parent operation-state did not provide a receiver to the child operation-state that implemented the `inlinable_receiver` concept, then the child operation state would not instantiate the specialisation and would instead fall back to instantiating the primary template that just stores the receiver as normal.
- If the parent operation-state provides a receiver that implements the `inlinable_reciever` concept *and* the child operation implements the specialisation for `inlinable_receiver` then we end up instantiating the child operation state specialisation that can avoid storing the receiver.

Note that while it is optional for operation-state implementations to implement this protocol, it is recommended that all operation-state implementations do so, in order to maximise the effectiveness of the optimisation.

3.2. Adding a helper for child operation-states (optional)

When defining the operation-state for a sender, it would be overly verbose for the author to have to duplicate their logic across two specialisations as defined above.

To allow encapsulating this optimisation and eliminating the duplication of code, we can factor out this facility into a helper CRTP base-class which is responsible for storing (or producing on demand) the receiver.

This paper proposes optionally adding the following helper class for operation-state authors to use to enable the optimisation in their implementations:

```

// In <execution> header
namespace std::execution
{
    template<typename Derived, receiver Receiver>
    class inlinable_operation_state {
    protected:
        explicit inlinable_operation_state(Receiver r)
            noexcept(std::is_nothrow_move_constructible_v<Receiver>)
            : rcvr_(std::move(r)) {}

        Receiver& get_receiver() noexcept { return rcvr_; }

    private:
        Receiver rcvr_; // exposition-only
    };

    template<typename Derived, receiver Receiver>
    requires inlinable_receiver<Receiver, Derived>
    class inlinable_operation_state<Derived, Receiver> {
    protected:

```



```

    explicit inlinable_operation_state(Receiver r) noexcept {}

    Receiver get_receiver() noexcept {
        return Receiver::make_receiver_for(static_cast<Derived*>(this));
    }
};
}

```

This class can then be used as a base-class of any operation-state that wants to be able to opt-in to this optimisation.

For example, the above `my_op_state` class can now be written as a single primary template by inheriting publicly from `inlinable_operation_state`:

```

template<typename Receiver>
class my_op_state : public inlinable_operation_state<my_op_state<Receiver>, Receiver> {
public:
    my_op_state(Receiver r) noexcept
        : inlinable_operation_state<my_op_state, Receiver>(std::move(r))
    {}

    void start() noexcept {
        // Call this->get_receiver() to get the receiver from the base-class.
        auto st = std::get_stop_token(std::execution::get_env(this->get_receiver()));
        if (st.stop_possible()) {
            // ...
        }
    }
};

```

This facility will be useful for all sender implementations (basically any sender that might become a child-operation of some sender-algorithm). This includes both leaf sender operations, which I expect will be the majority of senders authored by users, as well as sender-algorithms that compose other senders.

However, this facility is also fairly simple and straight-forward for users to write themselves when authoring sender implementations. It is only 20 lines of code and so the benefit from having such a facility in the standard library is one of convenience rather than one of abstracting away something complex that would be difficult to write by-hand.

3.3. Implementing `make_receiver_for()`

So, now that we have shown the child-operation part of the protocol and how it can use this protocol to avoid storing the receiver, let's now turn to looking at how we can actually implement this protocol from the parent-operation side.

This part of the protocol is considerably more involved, and there are a few pitfalls that we need to be careful to avoid, lest we unintentionally invoke undefined behaviour.

A naive first approach might be to try something like the following which uses `offsetof` to compute the address of the parent operation from the address of the child:

```

template<typename ParentReceiver, typename ChildSender>
class parent_op
    : public std::execution::inlinable_operation_state<parent_op<ParentReceiver, ChildSender>, ParentReceiver>
private:
    struct child_receiver {
        parent_op* op;
    };

```

```

template<typename ChildOp>
static child_receiver make_receiver_for(ChildOp* child_op) noexcept {
    static_assert(std::same_as<ChildOp, child_op_t>);
    // KEY PART: Compute address of parent_op from address of child_op
    auto* parent = reinterpret_cast<parent_op*>(
        reinterpret_cast<unsigned char*>(child_op) - offsetof(parent_op, child_op_));
    return child_receiver{parent};
}

// ... other receiver methods omitted for brevity
};

using child_op_t = std::connect_result_t<ChildSender, child_receiver>;
child_op_t child_op_;

public:
    parent_op(ChildSender&& child, ParentReceiver rcvr)
    : std::execution::inlinable_operation_state<parent_op, ParentReceiver>(std::move(rcvr),
    , child_op_(std::execution::connect(std::forward<ChildSender>(child), child_receiver{
    {}

    void start() noexcept {
        std::execution::start(child_op_);
    }
};

```

However, while this approach may appear to work on some implementations, *it is actually undefined behaviour* to do this.

It is not permitted to go from the address of a child data-member to the address of the parent class except in very limited circumstances. This rule is there to permit, among other things, a compiler-optimisation called "scalar replacement of aggregates", which allows the compiler to break up an aggregate type into a set of separate stack-allocations for each of the data-members if the address of the parent object is not aliased/observed.

The very limited circumstances in which we can go from the address of a sub-object to the address of the parent-object are the following:

- When the sub-object is a non-ambiguous base-class of parent-object ([\[expr.static.cast\] p11](#)) In this case, we can use `static_cast` to cast from pointer to base-class to the pointer to the derived parent-object
- When the parent-object and sub-object are "pointer-interconvertible" ([\[basic.compound\] p5](#)). In this case, we can use `reinterpret_cast` to cast from pointer to sub-object to pointer to parent-object.

Two objects are "pointer-interconvertible" only if:

- the parent-object is a union and the sub-object is a non-static data-member of that union; or
- the parent-object is a "standard layout" class object and the sub-object is the first non-static data-member of the parent-object or any base-class sub-object of the parent-object
- there exists an intermediate sub-object, C, such that the parent-object is pointer-interconvertible with C and C is pointer-interconvertible with the sub-object (i.e. the relationship is transitive)

Note that there are a number of rules for types that are considered "standard layout" class types ([\[class.prop\] p3](#)). I won't go into particular details here but, among other things, this doesn't allow types with virtual methods, virtual base-classes, types with non-static data-members with different access control, or data-members that are not also standard layout class types.

As child operation states in general are not going to all be standard layout types and since we also want to support cases where a parent-operation has multiple child operations, we cannot just rely on being able to convert the address of the first non-static data member to the address of the parent as a general solution.

This means that we are going to need to make use of base-classes to allow going from address of a sub-object to the address of a parent-object.

Further, there are also cases where we need to be able to defer construction of a child operation-state until after the operation is started, or where we want to be able to destroy a child operation-state before the parent operation-state is destroyed.

This means that, in general, we cannot just use the child operation-state as a direct base-class as this would force the lifetimes of the child operation-state to be the same as the lifetime of the parent operation-state. Instead, we can define a base-class that has as its only data-member an array of bytes which is used as storage for the child-operation state, into which we can placement-new the child operation-state at the appropriate time.

This can also be used to emulate unions of operation-states, where there might be a set of possible operation-state types that might need to be able to be constructed in that storage. For example, consider the set of possible operation-states for the successor operation of `let_value()`, the type of which may depend on what value completion-signature the predecessor completed with.

There are also some challenges with regards to avoiding circular dependencies when computing the complete type for the child operation-state. This will generally require the receiver type to be complete, but may also require the receiver's environment type to be complete if the child operation-state depends on the types of query-results (e.g. if it contains a stop-callback data-member).

However, as the layout of the child operation-state needs to be known during instantiation of a base-class of the parent operation-state type, the completeness of the receiver and its environment cannot depend on anything defined in the interface of the parent operation-state class.

This means that the return-types of all environment queries need to be known, even if the body of the query methods needs to access some state from the parent-operation-state (e.g. a stop-source). This information about the environment, therefore, needs to be injected into the base-class somehow, typically in the form of an additional template parameter.

Finally, since we might have multiple child operations which are constructed from the same sender (consider the child operations of `when_all(just(1), just(2))`), we need some way to distinguish different base-class child-objects so that we don't run into issues with duplicate base-classes, which would either be ill-formed or make the down-cast we want to perform ambiguous.

So, therefore, as we want to have a generic helper class we can use for the base-class, we also need to add some kind of 'tag' template parameter which can be passed something different for each child-operation to ensure that each child-operation base-class is distinct.

So, putting all of this together, we end up with some helper-classes like the following:

```
template<typename Sender, typename Receiver>
inline constexpr bool is_nothrow_connectable_v =
    noexcept(std::execution::connect(std::declval<Sender>(), std::declval<Receiver>()));

// Helper class for parent operations that want to manually manage the lifetime of
// a child operation.
template<typename ParentOp, typename Tag, typename Env, typename ChildSender>
class manual_child_operation_state {
private:
    class receiver {
    public:
        // Implement the prot
        template<typename ChildOp>
        static receiver make_receiver_for(ChildOp* child_op) noexcept {
            static_assert(std::same_as<ChildOp, child_op_t>);
```

```

// Cast from 'child_op_t*' to 'unsigned char*' pointer to 'storage_' member.
// - valid since we constructed at the storage address using placement-new.
auto* storage = reinterpret_cast<unsigned char*>(child_op);

// Cast from address of first member of 'manual_child_operation_state' to
// address of 'manual_child_operation_state'.
// Valid as 'manual_child_operation_state' is a standard-layout type.
auto* self = reinterpret_cast<manual_child_operation_state*>(storage);

// Cast from manual_child_operation_state address to address of 'ParentOp'
// which inherits from manual_child_operation_state.
auto* parent_op = static_cast<ParentOp*>(self);

// Construct a receiver with the address of the parent operation-state.
return receiver{parent_op};
}

// Forward following calls on the receiver to calls on the parent operation-state
// object with the added 'Tag' object as the first argument.

template<typename... Vs>
void set_value(Vs&&... vs) noexcept {
    parent_op->set_value(Tag{}, std::forward<Vs>(vs)...);
}

template<typename E>
void set_error(E&& e) noexcept {
    parent_op->set_error(Tag{}, std::forward<E>(e));
}

void set_stopped() noexcept {
    parent_op->set_stopped(Tag{});
}

Env get_env() const noexcept {
    return parent_op->get_env(Tag{});
}

private:
    friend manual_child_operation_state;
    explicit receiver(ParentOp* parent_op) noexcept : parent_op_(parent_op) {}
    ParentOp* parent_op_;
};

using child_op_t = std::execution::connect_result_t<ChildSender, receiver>;

protected:
// Trivial default constructor/destructor
manual_child_operation_state() noexcept = default;
~manual_child_operation_state() = default;

// Start execution of the child operation state.
void start() noexcept {
    std::execution::start(get());
}

// Manually construct the child operation from the sender.
void construct(ChildSender&& sender) noexcept(is_nothrow_connectable_v<ChildSender, r
    auto* parent_op = static_cast<ParentOp*>(this);
    ::new (&storage_) child_op_t(
        std::connect(std::forward<ChildSender>(sender), receiver{parent_op}));
}

// Manually destruct the child operation from the sender.

```

```

    void destruct() noexcept {
        get().~child_op_t();
    }

private:
    child_op_t& get() noexcept {
        return *std::launder(reinterpret_cast<child_op_t*>(&storage_));
    }

    alignas(child_op_t) unsigned char storage_[sizeof(child_op_t)];
};

// Helper class for parent operations that want a child operation with the same lifetime
// as that of the parent operation.
template<typename ParentOp, typename Tag, typename Env, typename ChildSender>
class child_operation_state : public manual_child_operation_state<ParentOp, Tag, Env, ChildSender>
private:
    using base_t = manual_child_operation_state<ParentOp, Tag, Env, ChildSender>;
    using base_t::construct;
    using base_t::destruct;

protected:
    explicit child_operation_state(ChildSender&& sender)
        noexcept(noexcept(base_t::construct(std::forward<ChildSender>(sender))) {
        base_t::construct(std::forward<ChildSender>(sender));
    }

    ~child_operation_state() {
        base_t::destruct();
    }
};

```

Revisiting the parent_op example above, it can now be rewritten as follows:

```

// A tag type to used for identifying which child a completion signal comes from
struct source_tag {};

template<typename ParentReceiver, typename ChildSender>
class parent_op
    : public std::execution::inlinable_operation_state<
        parent_op<ParentReceiver, ChildSender>,
        ParentReceiver>
    , public child_operation_state< // Inherit from 'child_operation_state'
        parent_op<ParentReceiver, ChildSender>,
        source_tag,
        std::execution::env_of_t<ParentReceiver>,
        ChildSender> {
    using inline_base_t = std::execution::inlinable_operation_state<parent_op, ParentReceiver>;
    using env_t = std::execution::env_of_t<ParentReceiver>;
    using child_base_t = child_operation_state<parent_op, source_tag, env_t, ChildSender>;

public:
    parent_op(ChildSender&& child, ParentReceiver rcvr)
        : inline_base_t(std::move(rcvr))
        , child_base_t(std::forward<ChildSender>(child))
    {}

    void start() noexcept {
        child_base_t::start();
    }

    //
    // Implement handling for signals coming from receiver passed to the

```

```

// 'source_tag' child operation.
//

template<typename... Vs>
void set_value(source_tag, Vs&&... vs) noexcept {
    // ...

    // Eventually... signal completion.
    std::execution::set_value(this->get_receiver(), the_result);
}

template<typename E>
void set_error(source_tag, E&& e) noexcept {
    // ...
}

void set_stopped(source_tag) noexcept {
    // ...
}

env_t get_env(source_tag) noexcept {
    return std::execution::get_env(this->get_receiver());
}
};

```

Some interesting points to note with this implementation:

- There will be a separate base-class for each child operation that is stored inline in the parent operation for which we want to be able to use this optimisation.
- We no longer need to define our own receiver class to pass to the child sender's connect method. This is all handled by the `child_operation_state` helper.
- The use of `inlinable_operation_state` means that this class can avoid storing the parent receiver if the parent operation state includes it as a sub-object, and the use of `child_operation_state` means that the child of this operation can avoid storing the receiver we pass to it if it uses the `inlinable_operation_state` class to manage storing (or not storing) the receiver. i.e. it implements the optimisation protocol both from the child-operation and parent-operation perspectives.
- All of the child completion signals are forwarded to methods on the operation-state, with signals from different children differentiated by a tag parameter.
- These methods need to be public to allow the `manual_child_operation_state::receiver` class to call them without having to declare it as a friend.
- This example just forwards through the parent environment to the child operation. If you wanted to modify the environment in some way (e.g. by changing the stop-token) then you'd need to define a separate environment class and pass that as the `Env` template argument to `child_operation_state` instead.

3.4. Adding a helper for parent operation-states (optional/future)

As evidenced by the long description above, it is complicated to try to implement the `make_receiver_for` function needed to enable the optimisation proposed by this paper.

There are a lot of subtle details that implementations of `make_receiver_for` need to get right and it's easy to accidentally run into undefined behaviour or to creating accidental cyclic dependencies that result in inscrutable compiler-errors.

Therefore, there is a reasonably high value in abstracting a lot of this away for users who want to write their own sender algorithms which implement the optimisation protocol proposed by this paper.

All users that want to implement their own sender algorithms that compose a known set of child operations would need such a facility if they wanted their algorithm to be able to participate in this optimisation.

However, such a facility would also be largely just an implementation detail for sender algorithms. The majority of users of the sender/receiver framework should be just composing those algorithms and, other than

As long as the implementers of sender-algorithms implement the protocol proposed by this paper in *some* way then users will benefit from the optimisations that are enabled by the protocol. Different libraries can use their own helper classes to implement the protocol - we do not need to standardise

3.5. Applying this optimisation to standard-library sender algorithms

In order for the optimisations proposed by this paper to be effective in wider code-bases, you generally want most of the algorithms you use to opt-in to the `inlinable_receiver` protocols, where possible.

A sender-adapter algorithm that does not opt-into the optimisation (either as a child or as a parent) will inhibit applying the optimisation at both the boundary with its children and at the boundary with its parent. Thus it will result in potentially adding two pointer-indirections in the middle of a sender expression.

So, as much as possible we want to make sure that standard-library senders all implement this optimisation.

The proposal P2300R10, which was merged into the draft standard, includes a number of sender factories and sender algorithms provided by the standard library.

Some of the algorithms have default implementations that are just compositions of other algorithms and so don't need any changes. These algorithms are:

- `starts_on()` - defined in terms of `let_value()` and `schedule()`
- `continues_on()` - defined in terms of `schedule_from()`
- `on()` - defined in terms of `write-env`, `continues_on` and `starts_on`.
- `stopped_as_optional()` - defined in terms of `let_stopped`, `then` and `just`.
- `stopped_as_error()` - defined in terms of `let_stopped`, and `just_error`.

The following algorithms are all of the algorithms which have some implementation of a sender for the default version of the algorithm that is not just a composition of other sender algorithms:

- `just`
- `just_error`
- `just_stopped`
- `read_env`
- `schedule_from`
- `then`
- `upon_error`
- `upon_stopped`
- `let_value`
- `let_error`
- `let_stopped`
- `bulk`
- `split`
- `when_all`
- `into_variant`
- `run_loop::run-loop-sender`

The design intent is to have each of these algorithms implement the optimization to avoid storing the receiver if the receiver connected to it satisfies `inlinable_receiver`. i.e. when this sender is used as the child of another operation that stores the child-operation as a sub-object.

Some of the above algorithms are leaf operations which do not have any children and so do not need to implement the `inlinable_receiver` concept themselves. These algorithms are: `just`, `just_error`, `just_stopped`, `read_env`, and `run_loop::run-loop-sender`.

The algorithms that do have children and thus would need to implement the parent operation side of the protocol are all of the other algorithms listed above.

The `run_loop::run-loop-sender` will need some individual rework to support omitting storage of the parent receiver, but this should be relatively straight-forward. The other algorithms are defined in terms of the exposition-only `basic-operation` and `basic-state` facilities and so should be able to have support added for omitting storage of the receiver in a generic way.

There are currently some assumptions in the design of the `impls-for<Tag>` interface that require the receiver object to exist for the duration of the `basic-state` object which will require some rework. For example, the `get-state` of `schedule_from` returns an object that holds a reference to the receiver. Similarly with `split`'s `get-state` function.

Implementing the parent-side of the optimisation protocol will require changes to move the child-operation states to be stored as base-classes rather than as the `basic-operation::inner-ops` tuple-like data-member.

The `let_value`, `let_error` and `let_stopped` algorithms all have an additional operation-state object stored in the object returned from `impls-for::get-state`. This object would also need to be moved to a base-class of `basic-operation`, but would need to have a manual lifetime and support being any of a set of possible operation-state types.

The `split` algorithm has a child operation that is a child of the `shared-state<Sndr>` structure. The child operation-state would need to be moved to a base-class and the `split-receiver<Sndr>` would need to be updated to define the `make_receiver_for()` static function.

All of this will need some major surgery to the specification machinery, but should not change the semantics of any of the existing algorithms.

4. Design Discussion

4.1. Naming of `inlinable_receiver` concept and `inlinable_operation_state`

The naming of the proposed concept `inlinable_receiver` and `inlinable_operation_state` base-class for operation-states both use the `inlinable` adjective to indicate that this is for operation-states which might be stored inline in their parent operation-state.

If we want to use a different name, for example because we don't want to use the term `inlinable` in this context, the following are some alternatives which could be considered.

Since a receiver that supports this concept is reconstructible from the operation-state address, it could use the name `reconstructible_receiver` or `reconstructible_receiver_from`, instead.

The other option is that we make the receiver concept exposition-only and only provide the `inlinable_operation_state` class as this would likely be the facility that most people would reach for rather than constraining their own class specializations on the concept.

With regards to naming of the `inlinable_operation_state` helper class, we could also choose a name that reflects better its purpose as a holder for the receiver by naming it `receiver_holder_base<Op, Rcvr>`, or similar.

5. Proposed Wording

Modify [execution.syn] as follows:

```
...
template<class Sch>
    concept scheduler = see below;

// [exec.recv], receivers
struct receiver_t {};

template<class Rcvr>
    concept receiver = see below;

template<class Rcvr, class Completions>
    concept receiver_of = see below;

template<class Rcvr, class ChildOp>
    concept inlinable_receiver =
        receiver<Rcvr> &&
        requires (ChildOp* child) {
            { Rcvr::make_receiver for(child) } noexcept -> same as<Rcvr>;
        };

struct set_value_t { unspecified };
struct set_error_t { unspecified };
struct set_stopped_t { unspecified };

inline constexpr set_value_t set_value{};
inline constexpr set_error_t set_error{};
inline constexpr set_stopped_t set_stopped{};

// [exec.opstate], operation states
struct operation_state_t {};
...
```

Add the following paragraph to [exec.recv.concepts] between p1 and p2:

The `inlinable_receiver` concept defines the requirements for a receiver that can be reconstructed on-demand from a pointer to the operation-state object created when the receiver was connected to a sender. Given a receiver object, `rcvr`, of type `Rcvr`, which was connected to a sender, producing an operation-state object, `op`, of type `Op`, and where `Rcvr` models `inlinable_receiver<Op>`, then the expression, `Rcvr::make_receiver for(addressof(op))`, evaluates to a receiver that is equal to `rcvr`.

[Note: Such a receiver does not need to be stored as a data-member of `op` as it can be recreated on demand - end note]

NOTE: The rest of the wording changes necessary for having the standard library async algorithms implement the optimisation will need some significant refactoring to remove some assumptions about the receiver object lifetime and will be provided in a subsequent revision of this paper.

6. References

- <https://github.com/cplusplus/sender-receiver/issues/224> Original github issue describing the problem and suggested solution described by this paper.